

Attention is all you need

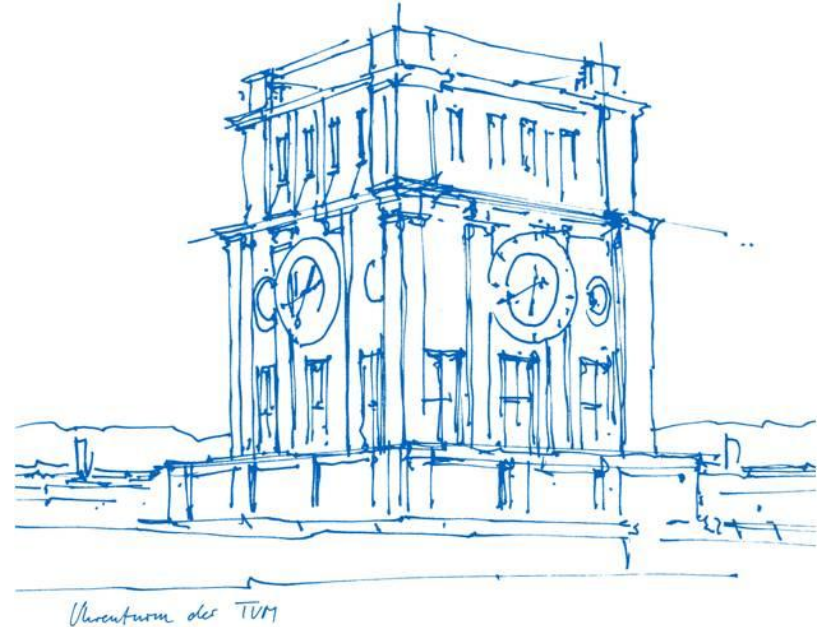
Technische Universität München

TUM School of Computation, Information and Technology

Lehrstuhl für Datenverarbeitung

January 16, 2025

Hesham Ghonim, Zeyad Shaheen, Walid Ghazouani,
Zhenghao Zhang, Izabella Pavlova, Mohammed Ghanemi



Outline

1. Introduction and Key Contributions
2. Transformer Architecture
3. Attention Mechanism
4. Impacts and Conclusion

Introduction and Key Contributions

Problems with RNN/LSTM:

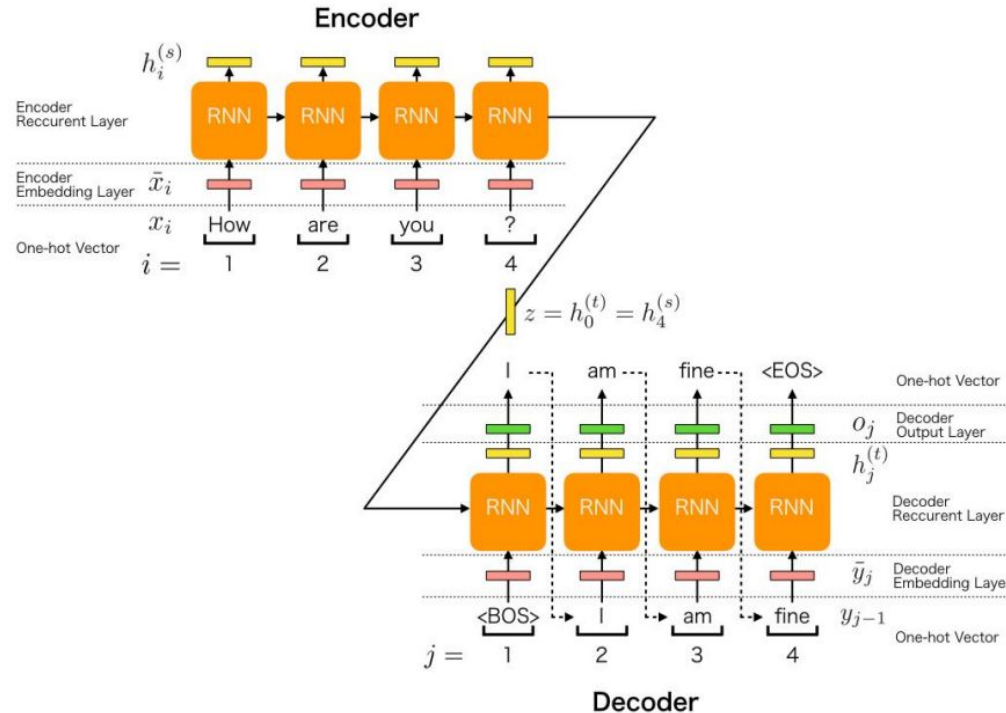
1. Struggle with long-range dependencies

In long sequences, RNNs often fail to connect distant words or elements, which negatively impacts the model's performance

2. Sequential processing limits parallelization

Since each step in an RNN depends on the previous one, it becomes difficult to leverage the full power of parallel processing hardware like GPUs/TPUs, leading to inefficiencies

3. Information would tend to be lost in bottleneck



Problems with RNN/LSTM:

1. Struggle with long-range dependencies
2. Sequential processing limits parallelization
3. Information would tend to be lost in bottleneck

The Transformer architecture:



- replaces **recurrence** with **self-attention**



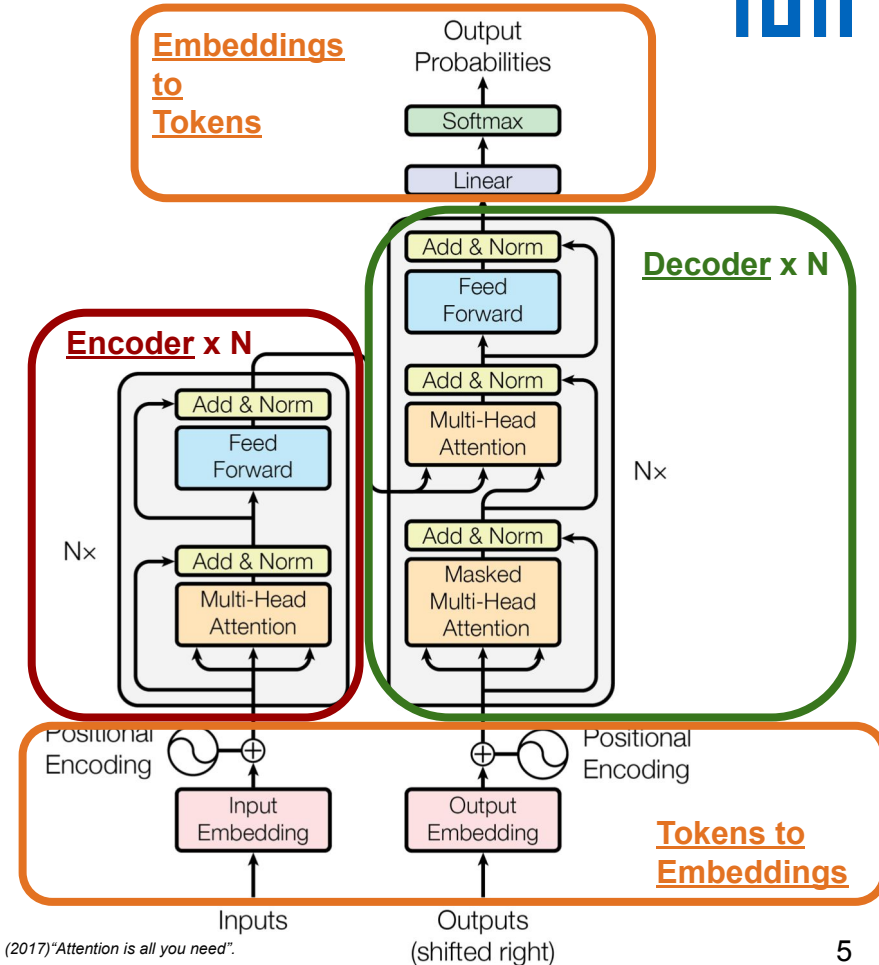
- enabling **parallel** processing



- replace **bottleneck** with **cross-attention**

Transformer Architecture

1. Tokens to Embeddings
2. Encoder
3. Decoder
4. Embeddings to Tokens



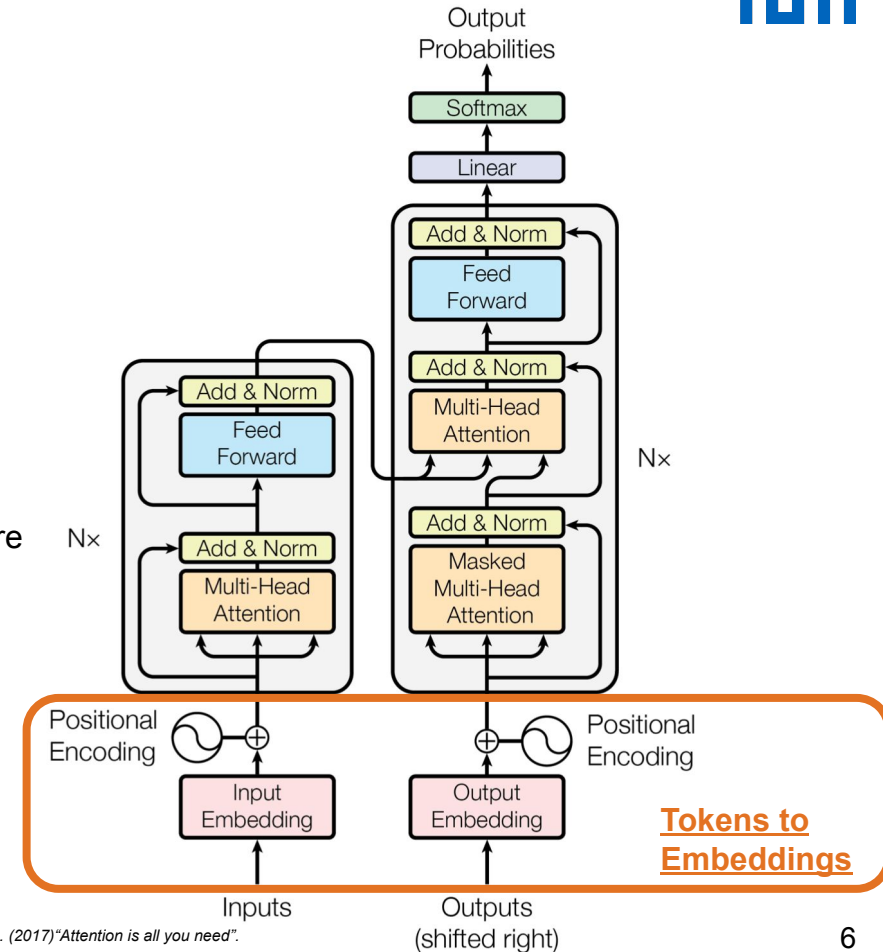
Transformer Architecture

Embedding Layer:

- convert the **input tokens** to **learned embeddings** of dimension d_{model} (512, 768...)

Positional Encoding:

- add **positional information** of tokens, as sequence orders are irrelevant within attention mechanism
- sinusoidal functions or learned positional embedding



Transformer Architecture

Multi-Head Self-Attention:

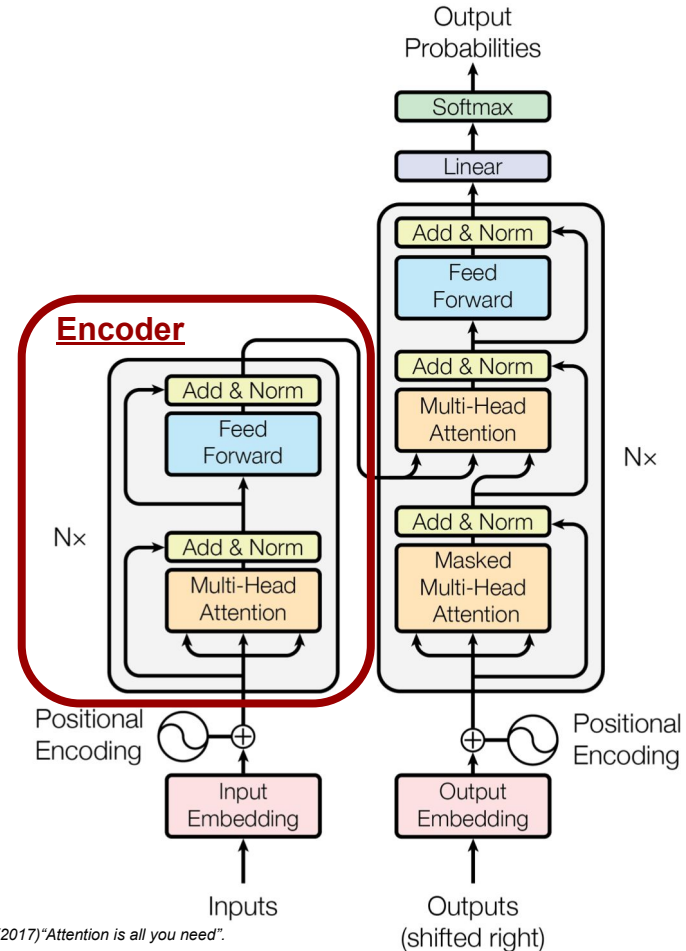
- using **multi-head attention mechanisms** to let each token capture **contextual information** from other tokens

Feed Forward Network:

- **fully connected network** with a residual connection
- usually only 1 hidden layer with a larger hidden dim

$$(4 \cdot d_{\text{model}})$$

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$



Transformer Architecture

Masked Multi-Head Self-Attention:

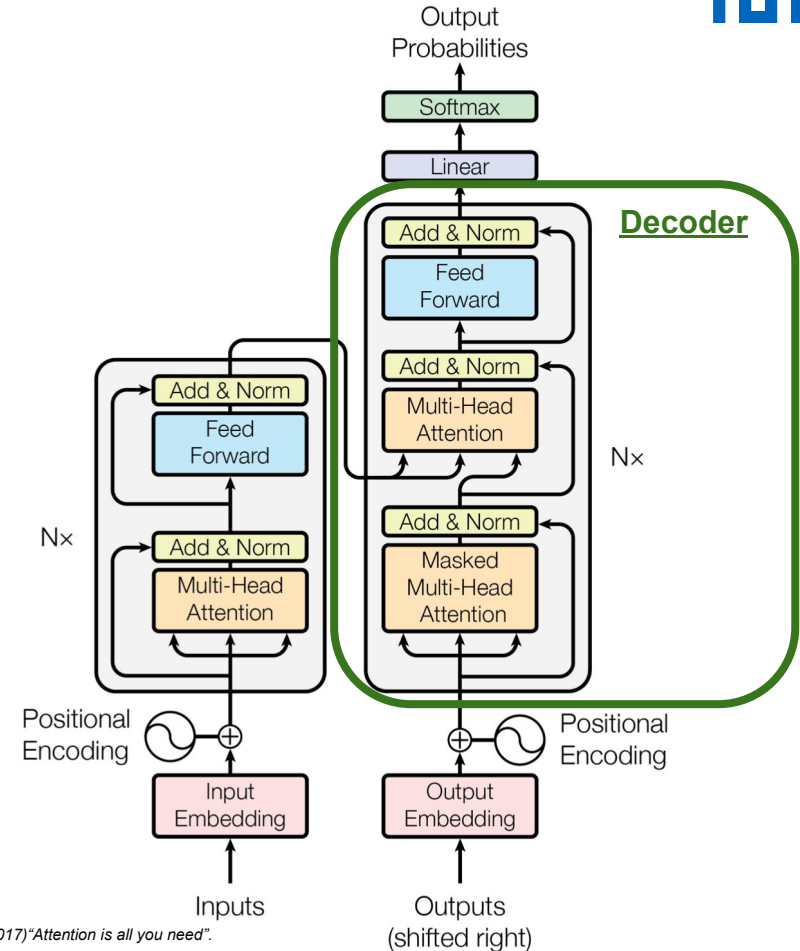
- similar to multi-head self attention in encoder
- mask for only see the tokens **in the past, not from future**

Multi-Head Cross-Attention:

- use the output from the **last encoder block** as **key, value**
- allows the decoder to see the whole input sequence

Feed Forward Network:

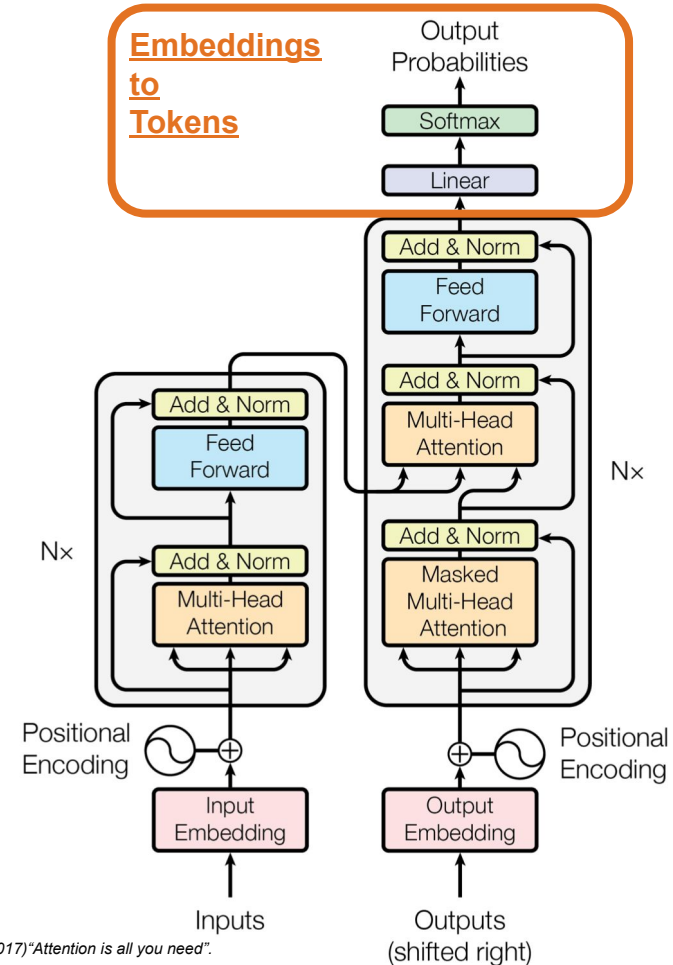
- similar to feed forward network in encoder



Transformer Architecture

Embedding2Token:

- convert **output embeddings** of dimension d_{model} to **token probabilities**
- choose the token with highest probability as the next predicted token



Self-Attention Mechanism

What is self-attention mechanism?

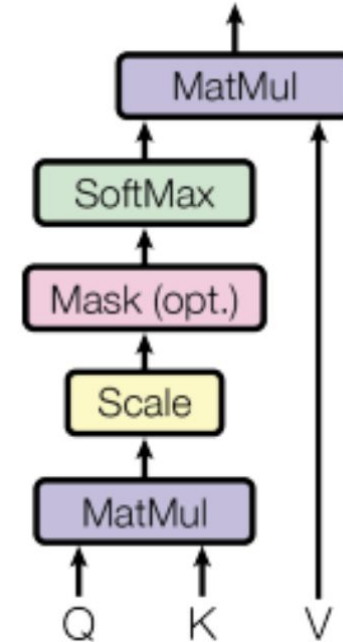
- a mechanism to compute relationships between all words in a sequence

Why is it important?

- captures long-range dependencies efficiently

How does it works?

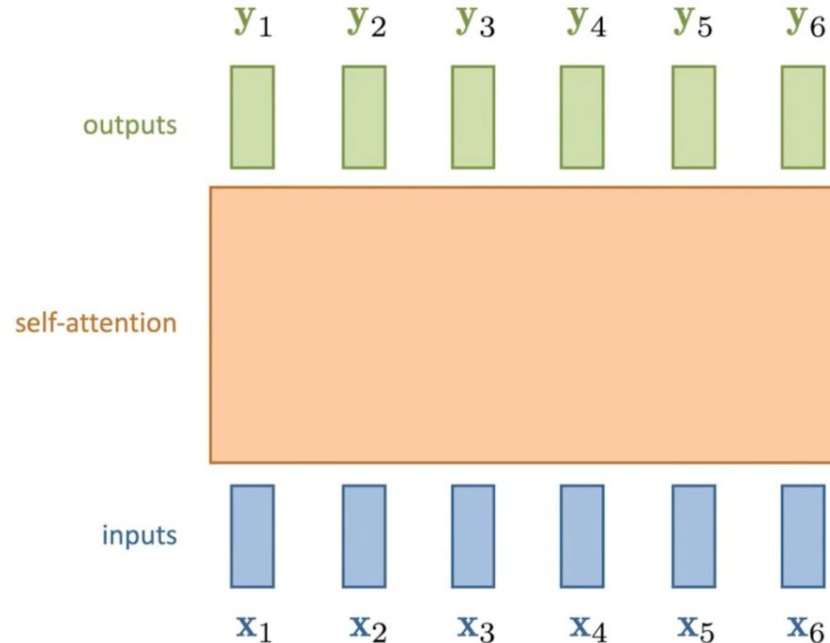
- computes attention scores for each word pair



Simple Self-Attention

Mechanism:

- for each output vector $\mathbf{y}$ we perform a weighted sum over the all input vectors
- $\mathbf{w}$ is not learned parameters, but a softmax of a dot product between two input vectors

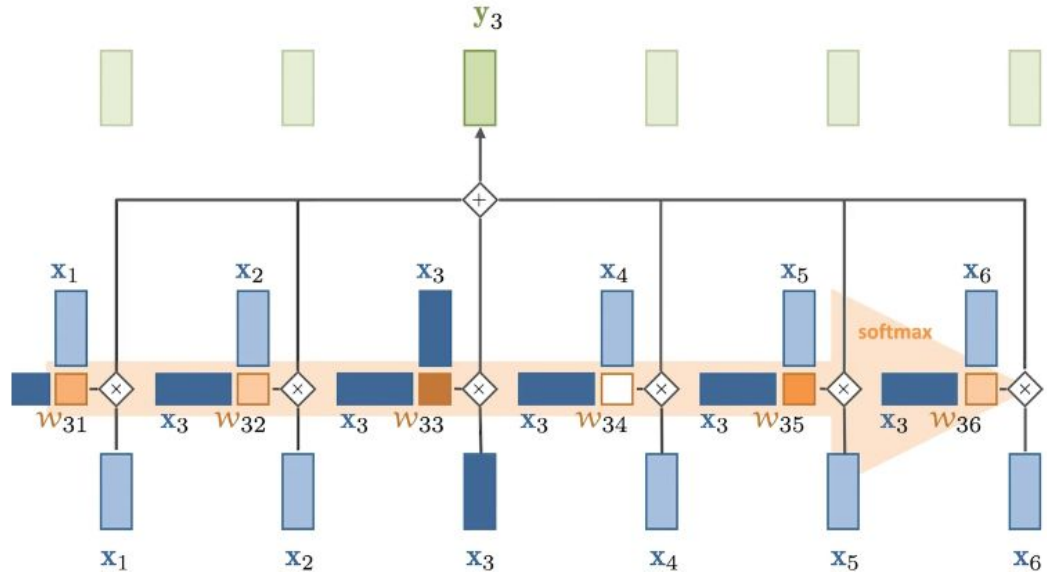


$$\mathbf{y}_i = \sum_j \mathbf{w}_{ij} \mathbf{x}_j$$

Simple Self-Attention

Mechanism:

- for each output vector $\mathbf{y}$ we perform a weighted sum over the all input vectors
- $\mathbf{w}$ is not learned parameters, but a softmax of a dot product between two input vectors



Self-Attention Mechanism

Query:

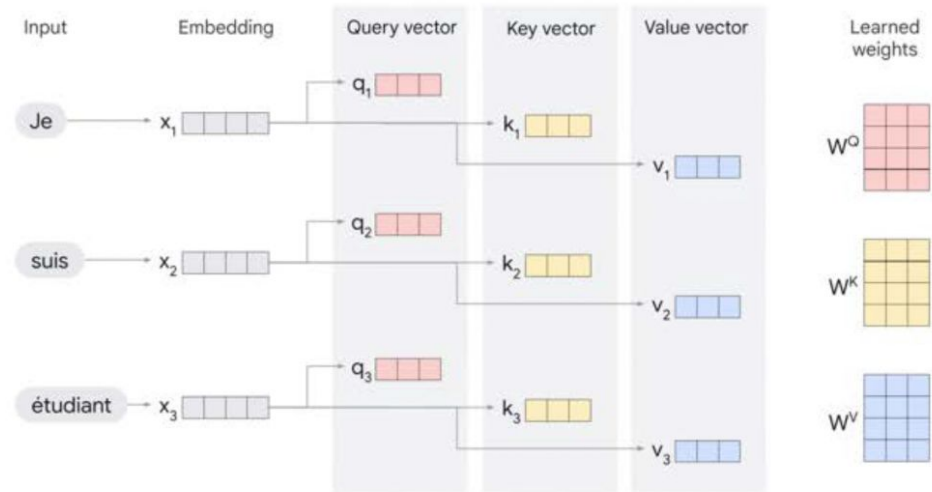
- What am I looking for?
- determines the relevance of other tokens to the current token

Key:

- What do I offer?
- represents the content of a token in a way that allows others to assess its relevance

Value:

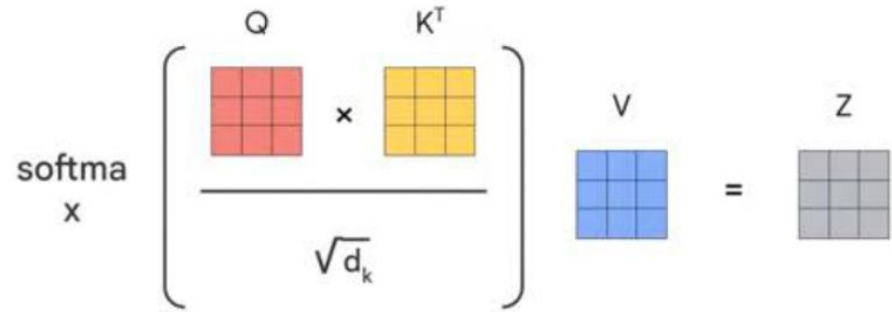
- What information do I have?
- contains the actual information that is aggregated based on the attention scores



Self-Attention Mechanism

Output:

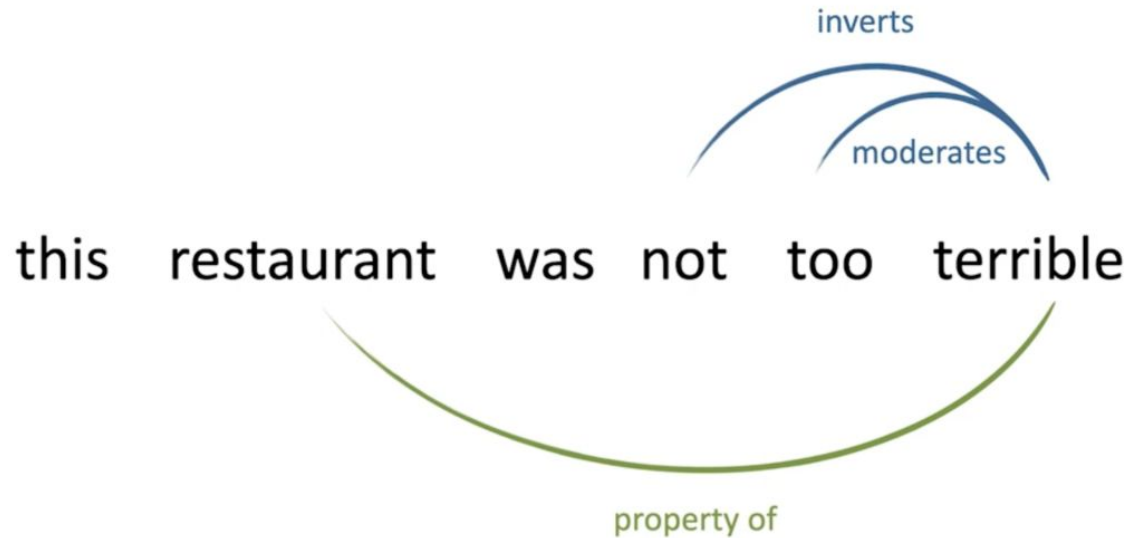
- the query vector of one token is compared to the key vectors of all tokens in the sequence via dot product
- the attention scores are normalized using the softmax function
- the attention weights are applied to the value vectors of all tokens



The diagram illustrates the Self-Attention Mechanism formula. It shows a red 3x3 matrix labeled 'Q' (Query) multiplied by a yellow 3x3 matrix labeled 'K^T' (Key Transposed). This product is divided by the square root of the dimensionality of the key vectors, $\sqrt{d_k}$. The result is then passed through a softmax function, indicated by 'softmax' and 'x' to the left of the large bracket. This weighted sum is then multiplied by a blue 3x3 matrix labeled 'V' (Value). The final result is a gray 3x3 matrix labeled 'Z' (Output).

$$\text{softmax}_x \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) \times V = Z$$

Multi-Head Attention



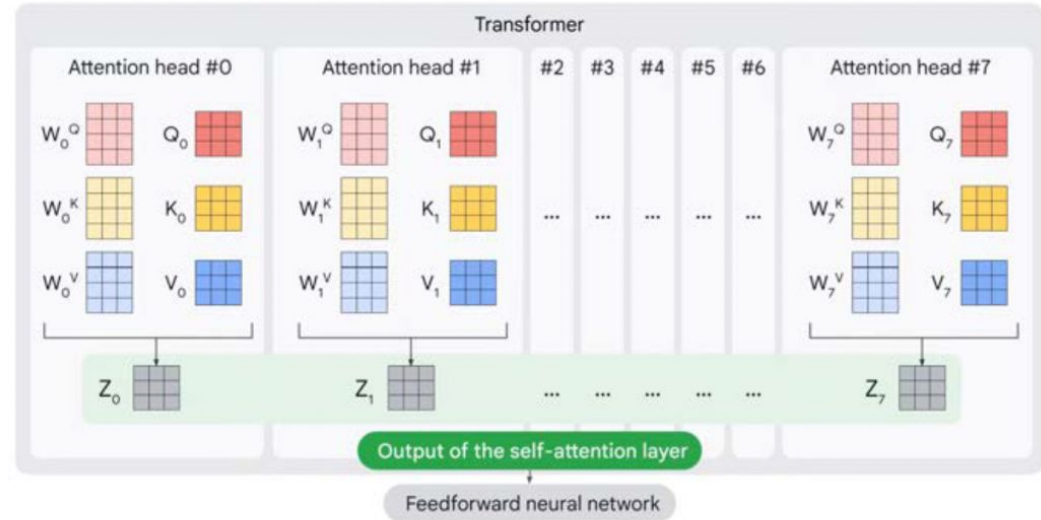
Multi-Head Attention

Key idea:

- compute multiple attention mechanism (heads) in parallel

Why to use it?

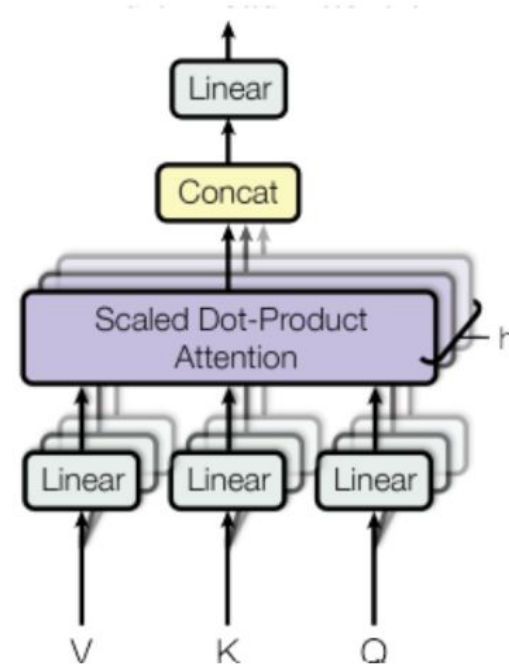
- each attention head can learn to focus on different aspects of the input (e.g. short / long-term dependencies, syntactic or semantic relationships)



Multi-Head Attention

Output:

- split the input embeddings into smaller subspaces for each head
- each head independently computes its own attention
- the outputs of all heads are concatenated and linearly transformed to produce the final result



Impacts and Conclusion

●: NLP
●: Computer Vision

Models	BERT	GPT	BART	Vision Transformer
Architecture	Encoder-only	Decoder-only	Encoder-decoder	Encoder-only
Key idea	Bidirectional training	Autoregressive generation	Bidirectional encoding and autoregressive decoding	Treats image patches as tokens and applies self-attention
Applications	Sentiment analysis, Q&A, and NER	Text generation tasks and chatbots	Translation, summarization	Image classification, object detection

Thank you for you Attention!

Q & A